



Lớp Animal / Dog / Cat — method overriding + super()

Cài đặt class Animal (base) với trường name, method make_sound() trả "generic sound", describe() trả "Tôi là {name}, tiếng kêu: {make_sound()}". Dog kế thừa Animal, override make_sound trả "Gâu!". Cat kế thừa Animal, override make_sound trả "Meo!". describe() của Dog/Cat **KHÔNG override** — vẫn dùng phiên bản của Animal, nhưng phải gọi **đúng** make_sound() của subclass (đa hình).

Python: mọi method đều dynamic dispatch sẵn. Khai báo class Dog(Animal): rồi định nghĩa lại make_sound(self) là xong.

C++: phải khai báo virtual std::string make_sound() const ở Animal (đã có trong skeleton) để C++ thực hiện đa hình. Trong Dog/Cat chỉ cần override std::string make_sound() const override. Driver dùng Animal* trỏ tới object con để polymorphism hoạt động (không copy về Animal — sẽ bị object slicing).

Bạn chỉ nộp khai báo lớp

File .py bạn nộp chỉ chứa **Animal + Dog + Cat** — **không** viết main, if __name__, hay driver đọc stdin. Grader **lấy tên file bạn nộp** làm tên module, **import lớp** của bạn, **chạy driver** để ném stdin vào và so sánh stdout.

 **Đặt tên file** là tên identifier hợp lệ (vd. Point.py, TuLanh.py, Animal.py). Nếu đặt bậy (my-bai.py, 1.py) grader fallback student_solution.

Luồng chấm bài

Animal.py (bạn nộp) → from Animal import * → driver grader → stdin → stdout → so khớp = pass

 Khung lớp bạn nộp lên

PythonC++

```
class Animal:

    def __init__(self, name: str):

        self.name = name
```

```

def make_sound(self) -> str:

    return 'generic sound'

def describe(self) -> str:

    return f'Tôi là {self.name}, tiếng kêu: {self.make_sound()}'

```

```

class Dog(Animal):

    # TODO: override make_sound() → 'Gâu!'

    ...

```

```

class Cat(Animal):

    # TODO: override make_sound() → 'Meo!'

    ...

```

```

class Animal {

protected:

    std::string name_;

public:

    Animal(const std::string& n) : name_(n) {}

    virtual ~Animal() {}

    virtual std::string make_sound() const { return "generic sound"; }

```

```

        std::string describe() const {

            return "Tôi là " + name_ + ", tiếng kêu: " + make_sound();

        }

};

class Dog : public Animal {

public:

    Dog(const std::string& n) : Animal(n) {}

    // TODO: override make_sound() → "Gâu!"

};

class Cat : public Animal {

public:

    Cat(const std::string& n) : Animal(n) {}

    // TODO: override make_sound() → "Meo!"

};

```

▢ Grader sẽ làm gì với lớp của bạn

1. Đọc N — số động vật
2. Mỗi dòng `kind name (dog / cat / animal)`
3. Tạo object tương ứng — Py: `Dog(name)/Cat(name)/Animal(name)`; C++: `new Dog(name)...` gán vào `Animal*`
4. In `describe()` — Py: `print(a.describe())` / C++: `std::cout << a->describe()`

► Xem code driver grader tự chạy

Grader chạy đoạn code sau cùng file bạn nộp. Dùng nút ở khung khai báo để xem driver cho từng ngôn ngữ:

```
from Animal import * # file SV nộp

n = int(input())

for _ in range(n):

    kind, name = input().split(maxsplit=1)

    a = {'dog': Dog, 'cat': Cat}.get(kind, Animal)(name)

    print(a.describe())
```

```
#include <iostream> // grader tự prepend khi compile

// ... + class bạn viết ...

int main() {

    int n; std::cin >> n; std::cin.ignore();

    for (int i = 0; i < n; ++i) {

        std::string line; std::getline(std::cin, line);

        size_t sp = line.find(' ');

        std::string kind = line.substr(0, sp);

        std::string name = line.substr(sp + 1);

        Animal* a = nullptr;

        if (kind == "dog") a = new Dog(name);
```

```
        else if (kind == "cat") a = new Cat(name);

        else a = new Animal(name);

        std::cout << a->describe() << std::endl;

        delete a;

    }

    return 0;
}
```

□ Testcase mẫu — để đối chiếu format

Grader chạy *code SV + driver* rồi so sánh stdout từng dòng (strip whitespace cuối, bỏ dòng trống thừa ở cuối). Các testcase ẩn có cùng định dạng.

Ví dụ 1

stdin

```
2

dog Lucky

cat Mio
```

expected stdout

```
Tôi là Lucky, tiếng kêu: Gâu!

Tôi là Mio, tiếng kêu: Meo!
```

Ví dụ 2

stdin

```
3

dog Rex
```

```
animal Unknown
```

```
cat Whiskers
```

expected stdout

```
Tôi là Rex, tiếng kêu: Gâu!
```

```
Tôi là Unknown, tiếng kêu: generic sound
```

```
Tôi là Whiskers, tiếng kêu: Meo!
```

Ngôn ngữ hỗ trợ: bài này nhận `.py/.cpp`. Grader tự ghép driver + class bạn nộp rồi chạy. Java sẽ mở khi có driver tương ứng.

□

Đây là bài **thực hành** — bạn **nộp lại bao nhiêu lần cũng được**. Mỗi lần nộp, hệ thống giữ điểm cao nhất. Thử, đọc error, fix, thử tiếp.